

Arquitetura do Site

Código comentado e decisões de implementação

Disciplina: Engenharia de Software

Acadêmicos: Arthur Ferreira dos Santos · Nickolay A J Dias · Gianluca Barraco Pimentel · Vinicyus Eduardo Vicentini Faria

Instituição: UniCesumar · 2026

Este documento descreve a arquitetura técnica do site desenvolvido como entrega da disciplina. O foco está em como HTML, CSS e JavaScript foram organizados, quais padrões foram aplicados e por que cada decisão foi tomada. O conteúdo do site (textos, APIs, exemplos) é apenas o tema escolhido — a lógica de construção é independente do assunto.

1. Estrutura geral: arquivo único

O site é entregue como um único arquivo HTML (index.html) com três blocos internos:

- `<style>` — todo o CSS: variáveis de tema, componentes, responsividade
- `<body>` — header, main (seções de conteúdo) e footer
- `<script>` — três blocos de JavaScript controlando a interatividade

Decisão e justificativa: por ser uma página estática hospedada no GitHub Pages, um único arquivo elimina requisições extras de CSS/JS separados, dispensa qualquer etapa de build e facilita edição colaborativa — qualquer integrante abre um arquivo e já vê o projeto inteiro.

1.1 Metadados do `<head>` e SEO básico

O bloco `<head>` contém três decisões de metadados relevantes:

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<meta name="description" content="ABOT é um projeto acadêmico que propõe
  automatizar buscas em plataformas digitais...">
<title>ABOT – Como funcionaria a automação de diagnóstico...</title>
```

- `charset UTF-8`: define a codificação de caracteres do documento. Sem isso, acentos e caracteres especiais do português podem aparecer corrompidos.
- `viewport`: instrui o navegador mobile a não reduzir a página para caber na tela — em vez disso, a página ocupa a largura real do dispositivo e o CSS responsivo cuida do layout. Sem essa tag, o site apareceria minúsculo em celulares mesmo com breakpoints definidos.
- `meta description`: texto exibido pelo Google abaixo do título nos resultados de busca. Não afeta o ranking diretamente, mas influencia a taxa de cliques. Descreve o conteúdo real da página em uma ou duas frases.
- `title`: o texto que aparece na aba do navegador e como título no Google. Deve ser descritivo e único para cada página.

2. Sistema de tema com variáveis CSS

Toda cor e fonte do site partem de um único bloco `:root`. Funciona como uma tabela de "design tokens" — valores nomeados que o restante do CSS consome via `var()`:

```
:root {
  /* paleta principal */
  --verde-escuro: #0E3B2E; /* cor primária da marca */
  --verde-bot: #27AE60; /* cor de acento / ação */
```

```

--verde-claro:      #E7F5EC; /* fundos de destaque */
--creme:            #F7F5F1; /* fundo alternado */
--branco:           #FFFFFF;
--texto:            #162420; /* texto principal */
--texto-suave:      #5B6B63; /* texto secundário */
--borda:            #E2E7E3;

/* cores de estado / semântica */
--vermelho:         #D85C5C;
--laranja:          #F2994A;
--azul:             #3B82C4;
--roxo:             #8B6FD9;

/* cores de marca de cada plataforma */
--marca-instagram: #C13584;
--marca-tiktok:    #111111;
--marca-youtube:   #FF0000;
--marca-google:    #4285F4;

/* cores do rodapé */
--footer-texto:     #9CC2B0;
--footer-texto-fraco: #7E9C8E;

/* tipografia */
--fonte-display:    "Unbounded", sans-serif; /* títulos */
--fonte-corpo:      "Sora", sans-serif;      /* texto */
--fonte-mono:       "JetBrains Mono", mono;  /* dados */
}

```

Como é consumido no restante do CSS:

```

.btn-primario { background: var(--verde-bot); color: var(--branco); }
footer a      { color: var(--footer-texto); }
.aba-plataforma.ativa { border-color: var(--verde-bot); }

```

Decisões e justificativas:

- Trocar a paleta inteira exige editar apenas o bloco `:root` — nenhuma outra regra precisa ser tocada.
- As cores de marca de cada plataforma (`--marca-instagram` etc.) foram mantidas em `:root` em vez de hardcoded no HTML. Isso garante que o sistema de design tokens cubra 100% das cores da interface.
- CSS nativo (custom properties) foi preferido a Sass/Less: navegadores modernos suportam nativamente, não há etapa de compilação e o arquivo permanece HTML puro, compatível com GitHub Pages sem pipeline.
- Três famílias tipográficas com papéis fixos (display / corpo / mono) evitam mistura inconsistente de fontes entre seções.

2.1 Sobrescrita de cores em seções com fundo escuro

Duas seções do site têm fundo escuro (verde-escuro): a seção de Arquitetura e a seção de Capacidades adicionais. Isso cria um problema: os painéis de conteúdo dentro dessas seções

herdam a cor branca do texto definida pelo pai, mas precisam manter o texto cinza (como no restante do site).

A solução é escopar regras de cor específicas ao container da seção, sobrescrevendo a herança apenas onde é necessário:

```
/* a seção .arquitetura define cor branca para tudo dentro dela */
.arquitetura { background: var(--verde-escuro); color: var(--branco); }

/* os painéis de capacidades ficam dentro de .arquitetura,
   mas precisam manter texto cinza — sobrescrevemos só o necessário */
#capacidades .painel-integracao.ativo { color: var(--texto); }
#capacidades .integracao-info h3      { color: var(--verde-escuro); }
#capacidades .integracao-info p,
#capacidades .tabela-mini td          { color: var(--texto-suave); }
```

Decisão e justificativa: as regras de sobrescrita estão escopadas a #capacidades (ID único) e aplicadas apenas ao painel .ativo — não aos painéis ocultos (display:none). Isso evita declarações de estilo aplicadas a elementos que não estão sendo exibidos, mantendo o CSS preciso e sem efeitos colaterais.

3. Componentes de UI reutilizáveis

Componentes de UI são os blocos visuais que compõem a interface — tudo que o usuário vê ou clica. Em vez de estilizar cada seção do zero, padrões visuais foram extraídos em classes reutilizáveis:

Classe	Onde é reutilizada
.btn + modificadoras	Botões do header, hero e CTA final. A classe base define layout e tipografia; .btn-primario, .btn-fora e .btn-grande definem variações de cor e tamanho
.cabecalho-secao	Bloco de título + subtítulo, repetido em todas as seções de conteúdo
.cartao	Cards da seção 'O problema' — grid de 3 colunas com hover consistente
.aba-plataforma	Botão de aba, reutilizado em duas seções independentes (Integrações e Capacidades)
.painel-integracao	Painel de conteúdo das abas, reutilizado nas mesmas duas seções
.eyebrow	Rótulo em maiúsculas acima dos títulos de seção, com fonte monoespçada
.diagrama-etapa	Etapas clicáveis do diagrama de arquitetura
.detalhe-etapa	Painel de detalhes revelado ao clicar em cada etapa

Exemplo — composição de classes no botão:

```

/* base: define layout, tipografia e transição */
.btn {
  display: inline-flex; align-items: center;
  border-radius: 6px; font-weight: 700;
  padding: 12px 24px; cursor: pointer;
  transition: transform .15s, box-shadow .2s, background .15s;
}

/* variações de cor */
.btn-primario { background: var(--verde-bot); color: var(--branco); }
.btn-fora      { border: 1.5px solid var(--verde-escuro); }

/* variação de tamanho */
.btn-grande    { padding: 16px 32px; font-size: 1rem; }

<!-- uso: combina classes conforme necessidade -->
<a class="btn btn-primario btn-grande">Ver como funcionaria</a>
<a class="btn btn-fora">Ver integrações</a>

```

Decisão e justificativa: separar a classe base das modificadoras segue o princípio de composição em vez de repetição — um novo botão herda o comportamento correto combinando classes existentes, sem duplicar CSS.

3.1 O logotipo em CSS puro

O símbolo da lupa que representa o ABOT no header e no rodapé é construído inteiramente com CSS — sem imagem, sem SVG e sem arquivo externo. É formado por quatro elementos sobrepostos via `position: absolute`:

```

/* container circular — o corpo da lupa */
.logo-mark {
  width: 34px; height: 34px;
  border-radius: 50%; /* torna o quadrado em círculo */
  background: var(--verde-escuro);
  position: relative; /* referência para os filhos absolutos */
}

/* anel interno — a lente da lupa */
.logo-mark::after {
  content: ""; /* pseudo-elemento precisa de content */
  position: absolute;
  width: 14px; height: 14px;
  border: 2px solid var(--branco);
  border-radius: 50%; /* anel circular */
  left: 6px; top: 6px;
}

/* haste da lupa — linha diagonal */
.logo-mark::before {
  content: "";
  position: absolute;
  width: 8px; height: 2px;
  background: var(--branco);
  transform: rotate(45deg); /* gira para formar a diagonal */
  right: 6px; bottom: 8px;
}

```

```
border-radius: 2px;
}

/* bolinha verde — o centro da lente */
.logo-mark span {
  width: 13px; height: 13px;
  border-radius: 50%;
  background: var(--verde-bot);
  position: absolute;
  left: 8.5px; top: 8.5px;
}
```

Decisão e justificativa: construir o logo em CSS elimina uma requisição de arquivo de imagem (que atrasaria o carregamento inicial) e garante que o logo seja perfeitamente nítido em qualquer resolução de tela — incluindo telas Retina de alta densidade, onde imagens rasterizadas (.png/.jpg) podem aparecer borradas. Pseudo-elementos `::before` e `::after` permitem adicionar dois elementos visuais extras a um único elemento HTML, sem precisar de markup adicional.

4. Layout em grid e responsividade

A maioria das seções usa CSS Grid para organizar colunas, com dois breakpoints que reduzem o número de colunas progressivamente:

```
/* Desktop: 4 colunas */
.diagrama { grid-template-columns: 1fr 1fr 1fr 1fr; }

/* Tablet — até 900px: 2 colunas */
@media (max-width: 900px) {
  .diagrama { grid-template-columns: 1fr 1fr; }
  .grid-3 { grid-template-columns: 1fr; }
  .painel-integracao.ativo { grid-template-columns: 1fr; }
  .footer-grid { grid-template-columns: 1fr 1fr; }
}

/* Mobile — até 600px: 1 coluna */
@media (max-width: 600px) {
  .diagrama { grid-template-columns: 1fr; }
  .footer-grid { grid-template-columns: 1fr; }
}
```

Decisões e justificativas:

- Apenas dois breakpoints cobrem desktop, tablet e celular — número mínimo necessário para o conteúdo permanecer legível.
- Todas as regras responsivas ficam nos dois blocos `@media` ao final do CSS, facilitando localizar e ajustar o comportamento mobile sem caçar regras espalhadas pelo arquivo.
- Elementos decorativos dependentes de posição (como a seta → entre etapas do diagrama) são ocultados em telas menores, pois perdem sentido quando os itens deixam de ficar em linha.

4.1 Header fixo (sticky) e navegação suave

O header permanece visível no topo da página enquanto o usuário rola o conteúdo, e os links de navegação levam suavemente à seção destino em vez de pular abruptamente:

```
/* header fica fixo no topo durante o scroll */
header {
  position: sticky;
  top: 0;
  z-index: 100;          /* garante que fique acima de qualquer conteúdo */
  background: var(--branco);
  border-bottom: 1px solid var(--borda);
}

/* scroll suave para os links de navegação interna (#ancora) */
html {
  scroll-behavior: smooth;
}
```

Decisões e justificativas:

- `position: sticky` mantém o header visível sem tirar o elemento do fluxo normal do documento — diferente de `position: fixed`, que remove o elemento do fluxo e pode causar sobreposição inesperada com o conteúdo abaixo.
- `z-index: 100` no header é o único `z-index` com propósito real no site: garante que o header fique acima de qualquer conteúdo que passe por baixo dele durante o scroll. É diferente do `z-index: 2` removido do diagrama na revisão, que não tinha nenhum elemento concorrente.
- `scroll-behavior: smooth` aplicado ao `html` faz com que todos os links de âncora (`#problema`, `#arquitetura`, `#integracoes` etc.) rolem suavemente até o destino em vez de pular instantaneamente, melhorando a experiência de navegação sem nenhum JavaScript.

5. Sistema de abas reutilizável e isolado

O site usa o mesmo padrão visual de abas em dois pontos distintos da página — Integrações e Capacidades adicionais — mas cada grupo funciona de forma completamente independente.

5.1 Marcação HTML

Cada grupo de abas usa um atributo `data-*` próprio para identificar o alvo, e os painéis usam o atributo correspondente para se identificar:

```
<!-- grupo 1: Integrações -->
<div id="abas-plataforma">
  <button data-plataforma="instagram">Instagram</button>
  <button data-plataforma="tiktok">TikTok</button>
</div>
<div data-painel="instagram"> ... </div>
<div data-painel="tiktok"> ... </div>
```

```

<!-- grupo 2: Capacidades -->
<div id="abas-capacidade">
  <button data-capacidade="bots">Bots autônomos</button>
</div>
<div data-capacidade-painel="bots"> ... </div>

```

5.2 JavaScript — ES6+ com const e blocos {}

Cada grupo é controlado por um bloco JavaScript independente. A versão atual usa const, let e arrow functions (ES6+) — sem IIFE nem var:

```

// Grupo 1 - Integrações por plataforma
{
  const abas = document.querySelectorAll('#abas-plataforma .aba-plataforma');
  const paineis = document.querySelectorAll('[data-painel]');

  abas.forEach(aba => {
    aba.addEventListener('click', () => {
      const alvo = aba.dataset.plataforma;
      abas.forEach(a => a.classList.toggle('ativa', a === aba));
      paineis.forEach(p => p.classList.toggle('ativo', p.dataset.painel === alvo));
    });
  });
}

// Grupo 2 - Capacidades adicionais (mesma lógica, atributos distintos)
{
  const abas = document.querySelectorAll('#abas-capacidade .aba-plataforma');
  const paineis = document.querySelectorAll('[data-capacidade-painel]');

  abas.forEach(aba => {
    aba.addEventListener('click', () => {
      const alvo = aba.dataset.capacidade;
      abas.forEach(a => a.classList.toggle('ativa', a === aba));
      paineis.forEach(p => p.classList.toggle('ativo', p.dataset.capacidadePainel
=== alvo));
    });
  });
}

```

Decisões e justificativas:

- const em vez de var: const e let declaram variáveis com escopo de bloco — ficam contidas dentro das chaves {} onde foram declaradas. Duas constantes chamadas abas em blocos diferentes não se conflitam, porque cada uma existe apenas no seu bloco.
- Blocos {} em vez de IIFE: a versão anterior usava funções autoexecutáveis (IIFE) para criar isolamento de escopo — necessário com var, que vaza para o escopo da função ou do script inteiro. Com const/let, um simples bloco {} já fornece o mesmo isolamento de forma mais legível e direta.
- Arrow functions (=>) em vez de function(): sintaxe mais concisa para callbacks. aba => { ... } é equivalente a function(aba){ ... } para este caso de uso.

- `classList.toggle(classe, condição booleana)`: ativa a classe no elemento que foi clicado (a === aba retorna true) e desativa nos demais (false), tudo em uma única passagem pelo `forEach` — sem `if/else` explícito.
- Seletores escopados por ID (`#abas-plataforma` vs `#abas-capacidade`): mesmo que ambos os grupos usem a classe visual `.aba-plataforma`, cada bloco JS só enxerga os elementos do seu próprio container — os dois sistemas nunca interferem um no outro.

6. Diagrama interativo de etapas

A seção 'Como funcionaria' usa um padrão similar ao de abas, mas aplicado a etapas numeradas de um diagrama. Clicar em uma etapa revela o painel de detalhes correspondente.

```
{
  const etapas = document.querySelectorAll('.diagrama-etapa');
  const detalhes = document.querySelectorAll('.detalhe-etapa');

  // função nomeada extraída para evitar duplicação entre os dois listeners
  const ativar = (etapa) => {
    const n = etapa.dataset.etapa;
    etapas.forEach(e => {
      const ativa = e === etapa;
      e.classList.toggle('ativo', ativa);
      e.setAttribute('aria-selected', ativa ? 'true' : 'false');
    });
    detalhes.forEach(d => d.classList.toggle('ativo', d.dataset.detalhe === n));
  };

  etapas.forEach(etapa => {
    etapa.addEventListener('click', () => ativar(etapa));
    etapa.addEventListener('keydown', ev => {
      if (ev.key === 'Enter' || ev.key === ' ') {
        ev.preventDefault();
        ativar(etapa);
      }
    });
  });
}
```

Decisões e justificativas:

- A função `ativar` foi extraída como uma `const arrow function` nomeada, em vez de repetir a mesma lógica dentro de dois `addEventListener` diferentes (`click` e `keydown`). Isso segue o princípio DRY (Don't Repeat Yourself).
- Suporte a teclado: as etapas do diagrama são `<div>` com `tabindex='0'` e `role='tab'`. O listener de `keydown` trata `Enter` e `Espaço` da mesma forma que um clique, tornando o componente navegável por teclado — comportamento que `<button>` forneceria nativamente, mas que foi adicionado explicitamente por se tratar de `<div>`.
- `aria-selected` é atualizado junto com a classe `.ativo`, mantendo o estado visual e o estado semântico (para leitores de tela) sempre sincronizados.

- A troca automática de etapas por timer (setInterval) foi removida durante o desenvolvimento porque interrompia a leitura — o usuário perdia a etapa que estava lendo. O clique manual garante que o visitante controle o ritmo.

7. Syntax highlight manual via spans + CSS

Syntax highlight é a coloração do código por tipo de elemento — palavras-chave, strings, comentários e métodos recebem cores diferentes para facilitar a leitura, como em editores de código.

Nos blocos de exemplo de API do site, isso foi feito manualmente com `<span>` e classes semânticas:

```
<!-- HTML de um bloco de código -->
<pre>
  <span class="chv">async function</span> buscarPost(postId, token) {
    <span class="com">// monta a URL com os campos desejados</span>
    <span class="chv">const</span> url = <span
class="str">'https://graph.facebook.com/...'<</span>;
    <span class="chv">const</span> res = <span class="chv">await</span> <span
class="met">fetch</span>(url);
    <span class="chv">return</span> res.<span class="met">json</span>();
  }
</pre>

/* CSS — cada classe recebe uma cor */
.integracao-codigo .com { color: #7E9C8E; } /* comentários → cinza */
.integracao-codigo .chv { color: #7FE0AC; } /* palavras-chave → verde */
.integracao-codigo .str { color: #F2D49A; } /* strings → amarelo */
.integracao-codigo .met { color: #A6D8FF; } /* métodos → azul */
```

Decisão e justificativa: bibliotecas como Prism.js ou highlight.js analisam o código dinamicamente e aplicam as classes automaticamente. Para este site, os exemplos são estáticos, conhecidos e em quantidade pequena — uma biblioteca adicionaria uma dependência externa e tempo de carregamento extra sem nenhum ganho real. O highlight manual é mais leve e mantém as cores alinhadas à paleta do restante do site.

8. Acessibilidade e marcação semântica

Decisões de marcação que não alteram a aparência, mas afetam como o site é interpretado por leitores de tela e navegação via teclado:

- Elementos semânticos HTML5 (`<header>`, `<nav>`, `<main>`, `<section>`, `<footer>`) comunicam a estrutura do documento independente do CSS.
- `role='tablist'`, `role='tab'` e `role='tabpanel'` nas etapas do diagrama: como são `<div>` (e não `<button>`), esses atributos informam aos leitores de tela que se trata de um padrão de abas.
- `tabindex='0'` nas etapas do diagrama: torna cada etapa acessível por teclado (Tab para navegar, Enter/Espaço para ativar).

- `aria-selected` atualizado pelo JavaScript sempre que uma etapa é ativada, mantendo o estado semântico sincronizado com o visual.
- `aria-controls` ligando cada etapa ao ID do painel que ela controla (detalhe-1, detalhe-2 etc.).

```
/* foco visível para navegação por teclado */
a:focus-visible,
button:focus-visible {
  outline: 2px solid var(--verde-bot);
  outline-offset: 2px;
}
```

Decisão e justificativa: `:focus-visible` mostra o indicador de foco apenas quando o usuário está navegando por teclado — não aparece ao clicar com mouse, evitando poluição visual, mas garantindo orientação clara para quem depende de teclado.

9. Tipografia via Google Fonts (CDN)

Três famílias tipográficas são carregadas via CDN, cada uma com um papel fixo definido em `:root`:

Variável	Fonte e papel
<code>--fonte-display</code>	Unbounded (pesos 500–800) — títulos e logotipo. Personalidade técnica e moderna, coerente com a proposta de automação
<code>--fonte-corpo</code>	Sora (pesos 400–700) — parágrafos, botões e textos de interface. Humanista, boa legibilidade em qualquer tamanho
<code>--fonte-mono</code>	JetBrains Mono (pesos 400–500) — selos, rótulos de dados e código. Reforça visualmente o caráter de 'bot' e automação

```
<!-- preconnect reduz a latência da primeira conexão com o CDN -->
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Unbounded:wght@500;600;700;800
&family=Sora:wght@400;500;600;700
&family=JetBrains+Mono:wght@400;500&display=swap" rel="stylesheet">
```

Decisão e justificativa: `preconnect` antecipa a conexão com os domínios do Google Fonts antes de o navegador precisar baixar as fontes, reduzindo a latência. `display=swap` garante que o texto apareça com uma fonte de fallback enquanto as fontes personalizadas carregam, evitando página em branco.

9.1 Tipografia fluida com `clamp()`

Os títulos h1 e h2 do site usam `clamp()` para o tamanho da fonte em vez de um valor fixo. Isso cria uma tipografia fluida — o tamanho aumenta e diminui proporcionalmente com a largura da tela, sem precisar de media queries para a fonte:

```
h1 {
  font-size: clamp(2.1rem, 5.2vw, 3.6rem);
  /*      ^^^^^^^^  ^^^^^^^^  ^^^^^^^^
             mínimo    fluido    máximo    */
}

h2 {
  font-size: clamp(1.6rem, 3.4vw, 2.4rem);
}
```

`clamp(mínimo, preferido, máximo)` recebe três valores:

- mínimo (2.1rem): o tamanho nunca fica menor que isso, mesmo em telas muito estreitas
- preferido (5.2vw): o tamanho ideal, proporcional à largura da tela (vw = viewport width). Em uma tela de 1000px, 5.2vw = 52px
- máximo (3.6rem): o tamanho nunca ultrapassa esse valor, mesmo em telas muito largas

Decisão e justificativa: sem `clamp()`, seria necessário definir um tamanho fixo de fonte e criar media queries para ajustá-lo em cada breakpoint. Com `clamp()`, o tamanho se adapta continuamente entre o mínimo e o máximo, resultando em títulos que ficam bem em qualquer largura de tela com uma única linha de CSS.

10. Revisão de código: bugs e incoerências corrigidos

O código passou por uma revisão sistemática antes da entrega. Os pontos identificados e corrigidos foram:

10.1 Modernização do JavaScript: `var/IIFE` → `const/blocos`

A versão inicial do JavaScript usava `var` e IIFEs (Immediately Invoked Function Expressions) para isolar o escopo de cada sistema de abas:

```
// versão anterior — ES5 com IIFE e var
(function() {
  var abas = document.querySelectorAll('...');
  abas.forEach(function(aba) { ... });
})();
```

Com `var`, qualquer variável declarada fora de uma função vaza para o escopo do script inteiro. A IIFE criava uma função para isolar esse escopo — solução necessária no JavaScript antigo.

Na versão atual, `const` e `let` têm escopo de bloco naturalmente. Um bloco `{}` é suficiente para o isolamento:

```
// versão atual — ES6+ com const e bloco
{
  const abas = document.querySelectorAll('...');
  abas.forEach(aba => { ... });
}
```

Resultado: código mais legível, sem funções desnecessárias, e compatível com todos os navegadores modernos.

Por que escolhemos ES6+ em vez de manter ES5:

- Todos os navegadores modernos suportam ES6 desde 2016 — e o site não tem requisito de compatibilidade com navegadores antigos como IE11, descontinuado em 2022.
- O código comunica a intenção de forma mais direta: `const abas` deixa claro que essa variável não vai mudar; `aba => { }` é mais conciso e visualmente mais leve que `function(aba){ }`.
- Eliminar as IIFEs remove uma camada de indireção que só existia para contornar uma limitação do `var`. Com `const`, essa camada é simplesmente desnecessária — o isolamento vem de graça com o bloco `{ }`.
- Qualquer desenvolvedor que abrir o código hoje vai reconhecer a sintaxe ES6+ como o padrão atual — manter ES5 seria como escrever em um dialeto que ninguém usa mais por escolha.

10.2 Diagrama sem acessibilidade de teclado

As etapas do diagrama eram `<div>` com `cursor:pointer` mas sem `tabindex` nem `role` — visualmente clicáveis, mas inacessíveis por teclado. As abas (que usam `<button>`) já respondiam a teclado nativamente. A inconsistência foi corrigida adicionando `role='tab'`, `tabindex='0'`, `aria-selected` e handler de `keydown`.

10.3 Lista de APIs desatualizada no diagrama

A etapa 03 ('Coleta') listava apenas 3 APIs, mas a seção de Integrações foi expandida para 6. A lista foi atualizada para incluir Meta Ads, Google Analytics e Search Console.

10.4 Cores de UI fora do sistema de variáveis

As bolinhas coloridas das abas usavam valores hex diretamente no HTML (`#C13584`, `#111`, `#FF0000` etc.), quebrando a premissa de centralização em `:root`. Todas foram extraídas para variáveis nomeadas (`--marca-instagram`, `--marca-tiktok`, `--marca-youtube`, `--marca-google`, `--laranja`, `--vermelho`).

10.5 `<tbody>` ausente nas tabelas

As 9 tabelas `.tabela-mini` não tinham `<tbody>` — HTML tolera a omissão, mas é estrutura incompleta e inconsistente com a tabela `.comparativo`, que já usava `<thead>` e `<tbody>`. O `<tbody>` foi adicionado em todas as 9.

10.6 Regra CSS aplicada a elementos ocultos

`#capacidades .painel-integracao{color:var(--texto)}` era aplicada aos 3 painéis simultaneamente, mas apenas 1 está visível por vez. A regra foi restringida a `.painel-integracao.ativo` para aplicar a cor somente ao painel exibido.

10.7 z-index sem propósito

A seta $\rightarrow$ entre etapas do diagrama (`.diagrama-etapa::after`) definia `z-index:2`, mas nenhum outro elemento declara `z-index` na mesma área. A propriedade não tinha efeito e foi removida.

10.8 Link de seção ausente no rodapé

A seção `#capacidades` existia na navegação do header mas não nos links do rodapé. O link foi adicionado ao rodapé para manter consistência de navegação.

10.9 Referência a conteúdo inexistente

O CTA final mencionava 'o Canvas completo', mas nenhuma seção ou link no site fornece acesso a esse conteúdo. A referência foi removida do texto.

Resumo das decisões técnicas (seções 1 a 10)

Tabela consolidada de todas as decisões de arquitetura e implementação descritas neste documento.

Decisão / recurso	Justificativa resumida
Arquivo único HTML+CSS+JS (s.1)	Zero build step; deploy direto no GitHub Pages; fácil leitura colaborativa
charset, viewport, meta description (s.1.1)	Codificação correta, layout mobile funcional e descrição para buscadores
Variáveis CSS em :root (s.2)	Paleta e tipografia ajustáveis em um ponto; sem cores 'mágicas' espalhadas
Cores de marca em :root (s.2)	Design tokens cobrem 100% das cores, inclusive indicadores de plataforma
CSS nativo em vez de Sass/Less (s.2)	Suporte nativo nos navegadores; sem compilador; arquivo permanece HTML puro
Sobrescrita de cor por escopo (s.2.1)	Herança de cor branca bloqueada apenas onde necessário, sem efeitos colaterais
Classes base + modificadoras (s.3)	Composição em vez de duplicação; novos botões herdam comportamento por padrão
Logo em CSS puro (s.3.1)	Sem arquivo externo; nitidez perfeita em telas Retina; pseudo-elementos ::before/::after
CSS Grid + 2 breakpoints (s.4)	Cobertura responsiva mínima e suficiente; regras media agrupadas ao final
position: sticky no header (s.4.1)	Header visível durante scroll sem remover do fluxo do documento
z-index: 100 no header (s.4.1)	Único z-index com propósito real: garante header acima do conteúdo em scroll
scroll-behavior: smooth (s.4.1)	Navegação suave entre seções sem JavaScript adicional
Sistema de abas com const e blocos {} (s.5)	Isolamento de escopo moderno; sem IIFE; sem var; dois grupos independentes
Atributos data-* para pareamento (s.5)	Separa lógica (JS) de estrutura (HTML); fácil de estender com novos painéis
classList.toggle(cls, bool) (s.5)	Ativa e desativa em uma única passagem pelo forEach; sem if/else explícito
Seletores escopados por ID (s.5)	Dois grupos de abas com mesma classe visual nunca interferem entre si

Função ativar() extraída (s.6)	Princípio DRY: lógica compartilhada entre click e keydown em um único lugar
Suporte a teclado no diagrama (s.6)	tabindex, role='tab', keydown — iguala acessibilidade ao sistema de abas
aria-selected sincronizado (s.6)	Estado visual (.ativo) e semântico (aria) sempre em sincronia
Syntax highlight manual (s.7)	Sem dependência externa; cores alinhadas à paleta; exemplos são estáticos
HTML semântico + :focus-visible (s.8)	Estrutura legível por leitores de tela; foco visível só na navegação por teclado
role/tabindex no diagrama (s.8)	Divs clicáveis informam leitores de tela sobre seu papel de aba
Google Fonts via CDN + preconnect (s.9)	Tipografia personalizada com latência reduzida; display=swap evita página em branco
clamp() nos títulos (s.9.1)	Tipografia fluida entre mínimo e máximo sem media queries para fonte
const/let em vez de var (s.10.1)	Escopo de bloco; sem vazamento; código mais legível e moderno
Blocos {} em vez de IIFE (s.10.1)	Isolamento com menos código; IIFE só era necessário por limitação do var
Arrow functions => (s.10.1)	Sintaxe mais concisa para callbacks; equivalente funcional ao function()
<tbody> explícito (s.10.5)	Estrutura completa e consistente em todas as tabelas do site
Regras .ativo escopadas (s.10.6)	Estilo aplicado apenas ao painel visível; evita declarações mortas
Decisão / recurso	Justificativa resumida
Arquivo único HTML+CSS+JS (s.1)	Zero build step; deploy direto no GitHub Pages; fácil leitura colaborativa
charset, viewport, meta description (s.1.1)	Codificação correta, layout mobile funcional e descrição para buscadores
Variáveis CSS em :root (s.2)	Paleta e tipografia ajustáveis em um ponto; sem cores 'mágicas' espalhadas
Cores de marca em :root (s.2)	Design tokens cobrem 100% das cores, inclusive indicadores de plataforma
CSS nativo em vez de Sass/Less (s.2)	Suporte nativo nos navegadores; sem compilador; arquivo permanece HTML puro
Sobrescrita de cor por escopo (s.2.1)	Herança de cor branca bloqueada apenas onde necessário, sem efeitos colaterais
Classes base + modificadoras (s.3)	Composição em vez de duplicação; novos botões herdam comportamento por padrão

Logo em CSS puro (s.3.1)	Sem arquivo externo; nitidez perfeita em telas Retina; pseudo-elementos <code>::before/::after</code>
CSS Grid + 2 breakpoints (s.4)	Cobertura responsiva mínima e suficiente; regras media agrupadas ao final
position: sticky no header (s.4.1)	Header visível durante scroll sem remover do fluxo do documento
z-index: 100 no header (s.4.1)	Único z-index com propósito real: garante header acima do conteúdo em scroll
scroll-behavior: smooth (s.4.1)	Navegação suave entre seções sem JavaScript adicional
Sistema de abas com const e blocos {} (s.5)	Isolamento de escopo moderno; sem IIFE; sem var; dois grupos independentes
Atributos data-* para pareamento (s.5)	Separa lógica (JS) de estrutura (HTML); fácil de estender com novos painéis
classList.toggle(cls, bool) (s.5)	Ativa e desativa em uma única passagem pelo <code>forEach</code> ; sem <code>if/else</code> explícito
Seletores escopados por ID (s.5)	Dois grupos de abas com mesma classe visual nunca interferem entre si
Função ativar() extraída (s.6)	Princípio DRY: lógica compartilhada entre click e keydown em um único lugar
Suporte a teclado no diagrama (s.6)	<code>tabindex</code> , <code>role='tab'</code> , <code>keydown</code> — iguala acessibilidade ao sistema de abas
aria-selected sincronizado (s.6)	Estado visual (<code>.ativo</code>) e semântico (<code>aria</code>) sempre em sincronia
Syntax highlight manual (s.7)	Sem dependência externa; cores alinhadas à paleta; exemplos são estáticos
HTML semântico + :focus-visible (s.8)	Estrutura legível por leitores de tela; foco visível só na navegação por teclado
role/tabindex no diagrama (s.8)	Divs clicáveis informam leitores de tela sobre seu papel de aba
Google Fonts via CDN + preconnect (s.9)	Tipografia personalizada com latência reduzida; <code>display=swap</code> evita página em branco
clamp() nos títulos (s.9.1)	Tipografia fluida entre mínimo e máximo sem media queries para fonte
const/let em vez de var (s.10.1)	Escopo de bloco; sem vazamento; código mais legível e moderno
Blocos {} em vez de IIFE (s.10.1)	Isolamento com menos código; IIFE só era necessário por limitação do <code>var</code>
Arrow functions => (s.10.1)	Sintaxe mais concisa para callbacks; equivalente funcional ao <code>function()</code>

<tbody> explícito (s.10.5)	Estrutura completa e consistente em todas as tabelas do site
Regras .ativo escopadas (s.10.6)	Estilo aplicado apenas ao painel visível; evita declarações mortas